

Unit – II

Java Swing

Swing Basics - Introduction to Swing architecture, Differences between AWT and Swing, Top-level containers: JFrame, JDialog, JApplet, Content pane & component hierarchy. Swing Components - JLabel, JButton, JTextField, JTextArea, JCheckBox, JRadioButton, JComboBox, JList, JTable, JTree, JSlider, JProgressBar.

Swing Basics

Swing is a part of Java Foundation Classes (JFC) used to create GUI applications in Java. It is an improvement over AWT.

AWT is used for creating GUI in Java. However, the AWT components are internally depends on native methods like C functions and operating system equivalent and hence problems related to portability arise (look and feel. Ex. Windows window and MAC window). And, also AWT components are heavy weight. It means AWT components take more system resources like memory and processor time.

Due to this, Java soft people felt it is better to redevelop AWT package without internally taking the help of native methods. Hence all the classes of AWT are extended to form new classes and a new class library is created. This library is called JFC (Java Foundation Classes).

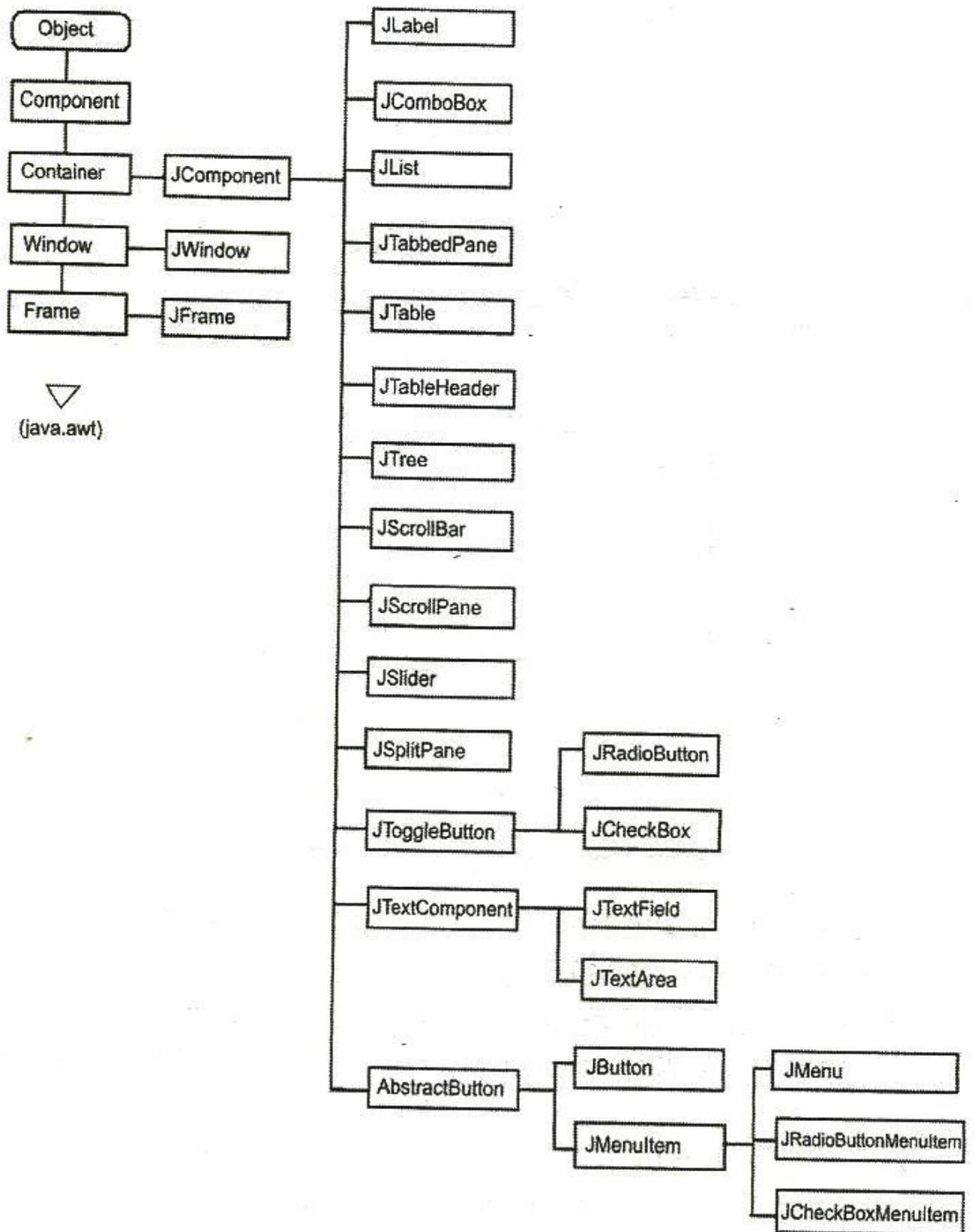
Features of Swing:

Feature	Description
Lightweight	Components are written in Java, not dependent on OS
Rich Set of Components	Buttons, labels, tables, trees, progress bars...
Pluggable Look and Feel	Change UI style at runtime
MVC Architecture	Separation of Model, View, Controller
Highly Customizable	Colors, borders, fonts can be easily changed

Basic Swing Components

Component	Description
JFrame	Window
JLabel	For displaying text
JButton	Push button
JTextField	Single-line input
JTextArea	Multi-line input
JPasswordField	Password entry
JCheckBox	Checkbox
JRadioButton	Radio button
JComboBox	Dropdown list
JList	List of items

Component	Description
JPanel	Container
JMenu	Menu items
JProgressBar	Shows progress
JScrollPane	For scrollable components



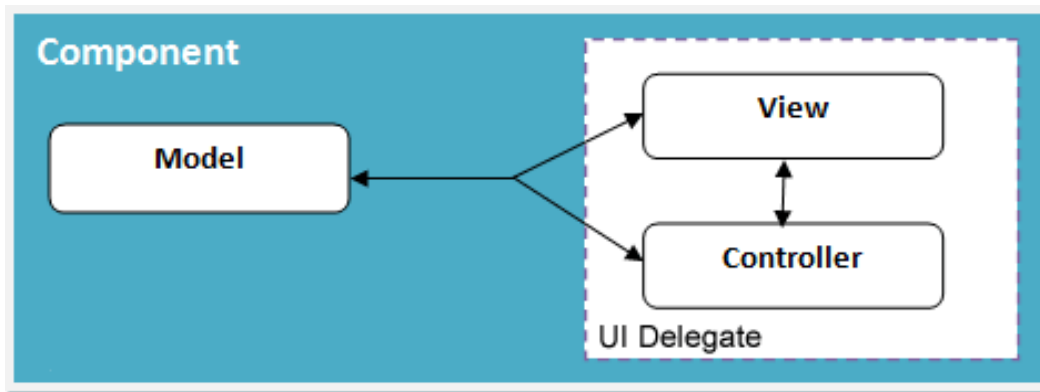
Introduction to Swing architecture

Java Swing follows a well-designed architecture called the Model–View–Controller (MVC) architecture. It provides a powerful way to build sophisticated and flexible GUI applications.

Swing architecture separates:

1. Model → Data
2. View → How data is displayed
3. Controller → How user interacts with the component

This structure makes Swing flexible, customizable, and extensible.



Swing uses a Modified (or loosely coupled) MVC pattern:

Model

- Holds data and state of the component
- Example:
 - `SpinnerModel` for `JSpinner`
 - `ListModel` for `JList`
 - `TableModel` for `JTable`
- Model is independent—same model can be used with different views.

View

- Responsible for visual representation (UI)
- What you see on the screen
- Swing components have a “UI Delegate” called Look and Feel (LAF)

Controller

- Handles user interactions
- Example: ActionListeners, MouseListeners, KeyListeners
- In Swing, controller logic is combined with the view (component itself)

Key Concepts in Swing Architecture

➔ Lightweight Components

Swing components are written in pure Java, not dependent on OS.

Examples: JButton, JTable, JTextField

➔ Pluggable Look and Feel

You can change the appearance of Swing components without changing code.

Example:

```
UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
```

➔ UI Delegates

Swing uses UI delegates, i.e., separate classes that handle how a component looks.

For example:

BasicButtonUI handles JButton appearance.

BasicTextFieldUI handles JTextField appearance.

➔ Event Dispatch Thread (EDT)

All Swing GUI operations must run on a single thread called the EDT.

Example:

```
SwingUtilities.invokeLater(() -> new MyFrame());
```

➔ Double Buffering

Swing uses double buffering to reduce flickering while drawing components.

Instead of drawing a component directly on the screen, Swing first draws it on a **hidden off-screen area (buffer)**.

When the drawing is complete, it **quickly shows the finished image on the screen**.

Difference Between AWT and Swing

S.No	AWT (Abstract Window Toolkit)	Swing
1	Uses native OS components	Written in pure Java (no OS dependency)
2	Components are heavyweight	Components are lightweight
3	Limited set of GUI components	Rich set of advanced components (JTable, JTree, etc.)
4	No pluggable look and feel	Supports pluggable Look & Feel (Metal, Nimbus, Windows)
5	Less flexible and customizable	Highly customizable
6	No strict MVC architecture	Follows Modified MVC architecture

S.No	AWT (Abstract Window Toolkit)	Swing
7	More prone to flickering	Uses double buffering , less flicker
8	Slower for modern GUIs	Faster and smoother graphics
9	Package: java.awt	Package: javax.swing
10	Components: Button, Label, TextField	Components: JButton, JLabel, JTextField, JPasswordField, JTable, etc.
11	Limited event handling	Advanced event handling through EDT
12	Older technology	Newer, more powerful, widely used
13	OS-dependent look	Same look across all platforms unless changed
14	Not suitable for complex UI	Suitable for modern, feature-rich UI
15	Example: Button b = new Button("OK")	Example: JButton b = new JButton("OK")

Top-level containers in swing

A Swing GUI consists of two key items: components and containers. However, this distinction is mostly conceptual because all containers are also components. The difference between the two is found in their intended purpose: As the term is commonly used, a component is an independent visual control, such as a push button or slider. A container holds a group of components. Thus, a container is a special type of component that is designed to hold other components.

Furthermore, in order for a component to be displayed, it must be held within a container. Thus, all Swing GUIs will have at least one container. Because

containers are components, a container can also hold other containers. This enables Swing to define what is called a containment hierarchy, at the top of which must be a top-level container.

Components:

In general, Swing components are derived from the JComponent class.

JComponent

provides the functionality that is common to all components. For example,

JComponent

supports the pluggable look and feel. JComponent inherits the AWT classes

Container and

Component. All of Swing's components are represented by classes defined within the

package javax.swing.

Containers:

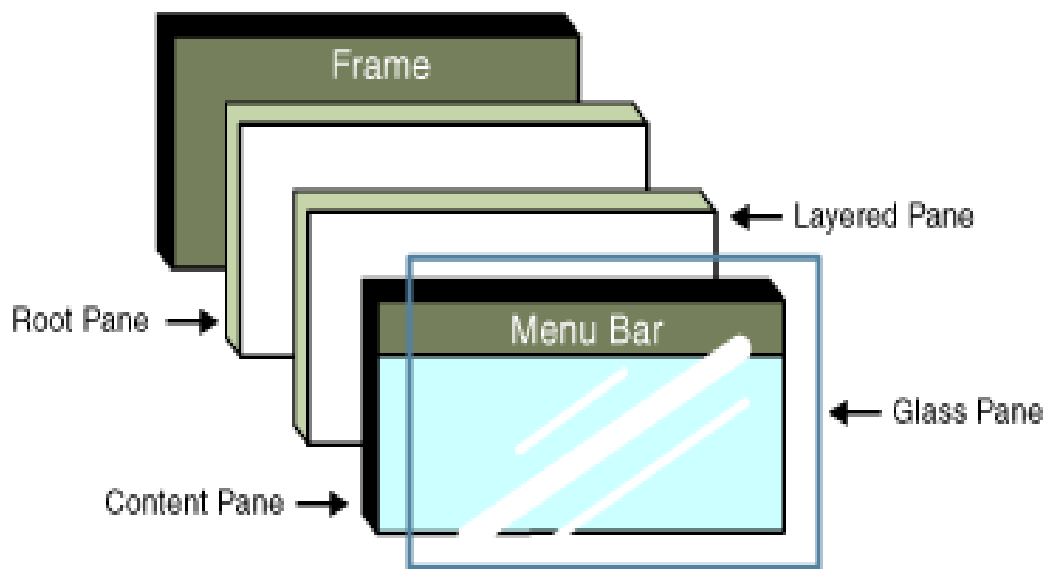
Swing defines two types of containers.

1. Top-level containers/ Root containers: JFrame, JApplet, JWindow, and JDialog. As the name implies, a top-level container must be at the top of a containment hierarchy. A top-level container is not contained within any other container.

Furthermore, every containment hierarchy must begin with a top-level container. The one most commonly used for applications are JFrame and JApplet. Unlike Swing's other components, the top-level containers are heavyweight. Because they inherit AWT classes Component and Container.

Whenever we create a top level container four sub-level containers are automatically created:

- Glass pane (JGlass)
- Root pane (JRootPane)
- Layered pane (JLayeredPane)
- Content pane



Glass pane:

This is the first pane and is very close to the monitor's screen. Any components to be displayed in the foreground are attached to this glass pane. To reach this glass pane we use `getGlassPane()` method of `JFrame` class, which return `Component` class object.

Root Pane:

This pane is below the glass pane. Any components to be displayed in the background are displayed in this frame. To go to the root pane, we can use `getRootPane()` method of `JFrame` class, which returns `JRootPane` object.

Layered pane:

This pane is below the root pane. When we want to take several components as a group, we attach them in the layered pane. We can reach this pane by calling `getLayeredPane()` method of `JFrame` class which returns `JLayeredPane` class object.

Content pane:

- This is bottom most of all, Individual components are attached to this pane. To reach this pane, we can call `getContentPane()` method of `JFrame` class which returns `Container` class object.
- All components like buttons, labels, textfields are added **to the content pane**, not to the `JFrame` directly.

Before Java 5

We had to write:

```
frame.getContentPane().add(button);
```

After Java 5

Swing allows:

```
frame.add(button);
```

Although you wrote `frame.add(button)`, Swing automatically interprets this as:

```
frame.getContentPane().add(button);
```

So the button is added to the content pane of the `JFrame`.

Example Program for Content Pane + Hierarchy

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
public class ContentPaneExample {
```

```
    public static void main(String[] args) {
```

```
        JFrame frame = new JFrame("Content Pane Example");
```

```
        // Frame settings
```

```
        frame.setSize(400, 300);
```

```
        frame.setLayout(null);
```

```
        frame.getContentPane().setBackground(Color.YELLOW);
```

```
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

// Creating a panel

```
JPanel panel = new JPanel();  
  
panel.setBounds(50, 50, 200, 100);  
  
panel.setBackground(Color.GREEN);
```

// Adding components to panel

```
JButton btn1 = new JButton("Button 1");  
  
JButton btn2 = new JButton("Button 2");  
  
panel.add(btn1);  
  
panel.add(btn2);
```

// Adding panel to content pane

```
frame.getContentPane().add(panel);  
  
frame.setVisible(true);  
  
}  
  
}
```

Output:



2. Lightweight containers – containers do inherit JComponent. An example of a

lightweight container is JPanel, which is a general-purpose container.

Lightweight

containers are often used to organize and manage groups of related components.

In Swing, every GUI application must start with a **top-level container**. These containers can hold other Swing components such as JButton, JLabel, JTextField, etc.

public int getWidth ()	Returns the current width of this component in pixel.
public int getHeight ()	Returns the current height of this component in pixel.
public int getX()	Returns the current x coordinate of the component's top-left corner.
public int getY ()	Returns the current y coordinate of the component's top-left corner.
public java.awt.Graphics getGraphics()	Returns this component's Graphics object you can draw on. This is useful if you want to change the appearance of a component.
public void setBackground (java.awt.Color bg)	Sets this component's background color.
public void setEnabled (boolean enabled)	Sets whether or not this component is enabled.
public void setFont (java.awt.Font font)	Set the font used to print text on this component.
public void setForeground (java.awt.Color fg)	Set this component's foreground color.
public void setToolTipText(java.lang.String text)	Sets the tool tip text.
public void setVisible (boolean visible)	Sets whether or not this component is visible.

In Swing, every GUI application must start with a top-level container. These containers can hold other Swing components such as JButton, JLabel, JTextField, etc.

Swing provides three top-level containers:

1. JFrame
2. JDialog
3. JApplet (deprecated but still in syllabus)

Each has its own purpose and usage.

1. JFrame:

Frame represents a window with a title bar and borders. Frame becomes the basis for creating the GUIs for an application because all the components go into the frame. To create a frame, we have to create an object to JFrame class in swing as

JFrame jf=new JFrame(); // create a frame without title

JFrame jf=new JFrame("title"); // create a frame with title

Common Methods:

Method	Description
setTitle()	Sets window title
setSize()	Sets size
setDefaultCloseOperation()	Defines closing behavior
add()	Adds components

To close the frame, use setDefaultCloseOperation() method of JFrame class

setDefaultCloseOperation(constant)

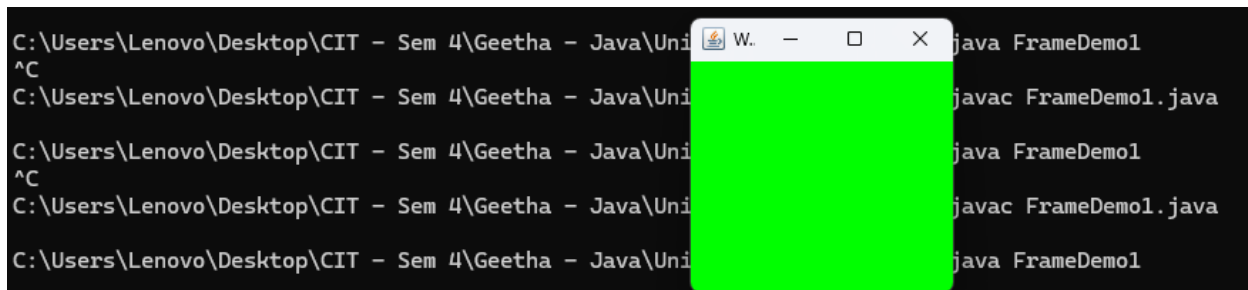
where constant values are

JFrame.EXIT_ON_CLOSE	This closes the application upon clicking the close button
JFrame.DISPOSE_ON_CLOSE	This closes the application upon clicking the close button
JFrame.DO_NOTHING_ON_CLOSE	This will not perform any operation upon clicking close button
JFrame.HIDE_ON_CLOSE	This hides the frame upon clicking close button

Example Program:

```
import javax.swing.*;
import java.awt.*;

class FrameDemo1
{
    public static void main(String arg[])
    {
        JFrame jf=new JFrame("Welcome");
        jf.setSize(200,200);
        jf.setVisible(true);
        jf.setDefaultCloseOperation(JFrame.HIDE_ON_CLOSE );
        Container c=jf.getContentPane();
        c.setBackground(Color.green);
    }
}
```



```
C:\Users\Lenovo\Desktop\CIT - Sem 4\Geetha - Java\Uni... java FrameDemo1
^C
C:\Users\Lenovo\Desktop\CIT - Sem 4\Geetha - Java\Uni... javac FrameDemo1.java
C:\Users\Lenovo\Desktop\CIT - Sem 4\Geetha - Java\Uni... java FrameDemo1
^C
C:\Users\Lenovo\Desktop\CIT - Sem 4\Geetha - Java\Uni... javac FrameDemo1.java
C:\Users\Lenovo\Desktop\CIT - Sem 4\Geetha - Java\Uni... java FrameDemo1
```


2. JDialog

JDialog is a pop-up window used to display **messages, confirmation boxes, or custom forms**.

Types of Dialogs:

- **Modal** → Blocks the parent window
- **Non-modal** → Allows interaction with parent window

Uses:

- Alerts, confirmation screens
- Login dialogs
- Settings popup

Example Program for JDialog:

```
import javax.swing.*;
import java.awt.event.*;

public class SimpleDialog {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Main Window");
        frame.setSize(300, 200);
        frame.setLayout(null);

        JButton button = new JButton("Open Dialog");
        button.setBounds(80, 60, 150, 30);
        frame.add(button);
```

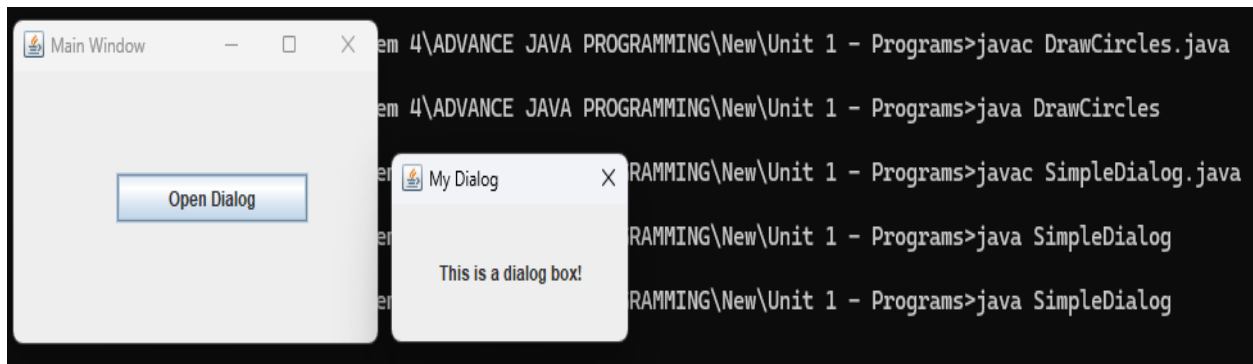
```

button.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        JDialog dialog = new JDialog(frame, "My Dialog", true);
        dialog.add(new JLabel("This is a dialog box!",
SwingConstants.CENTER));
        dialog.setSize(200, 120);
        dialog.setVisible(true);
    }
});

frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
}

```

Output:



3. JApplet

JApplet is a Swing-based applet that runs inside a web browser or applet viewer. (Works only in environments still supporting applets.)

Methods:

- `init()`
- `start()`
- `stop()`
- `destroy()`

Example Program for JApplet (MyApplet.java)

```
import javax.swing.*;
import java.applet.*;
import java.awt.*;

public class MyApplet extends JApplet {
    public void init() {
        // Set layout
        setLayout(new FlowLayout());

        // Create Swing components
        JLabel label = new JLabel("Hello! This is a simple JApplet.");
        JButton button = new JButton("Click Me");

        // Add components to applet
        add(label);
```

```
        add(button);  
    }  
}
```

HTML File to Run the Applet (MyApplet.html)

```
<html>  
<body>  
    <applet code="MyApplet.class" width="300" height="200">  
    </applet>  
</body>  
</html>
```

Step 1: Save the Java File (MyApplet.java)

Step 2: Compile the Java file - javac MyApplet.java

This will generate: MyApplet.class

Step 3: Create an HTML file (MyApplet.html)

Step 4: Run using Applet Viewer - appletviewer MyApplet.html

This will open the JApplet window.

Swing Components

Java Swing is a part of **Java Foundation Classes (JFC)** used to build **graphical user interfaces (GUI)**. Swing provides a rich set of lightweight components such as labels, buttons, text fields, menus, and scroll bars. Below are the commonly used Swing components.

JLabel

- Used to display **text or images**.
- It is **not editable** by the user.
- Commonly used for headings, instructions, names, etc.
- JLabel is used to display a text
 - JLabel(string str)
 - JLabel(Icon i)
 - JLabel(String s, Icon i, int align)
 - CENTER, LEFT, RIGHT, LEADING, TRAILING
- Icon – is an interface
 - The easiest way to obtain icon is to use ImageIcon class. ImageIcon class implements Icon interface.

Important Methods:

```
Icon getIcon()  
String getText()  
void setIcon(Icon icon)  
void setText(String s)
```

Example Program:

```
import javax.swing.*;  
  
public class JLabelIconExample {  
    public static void main(String[] args) {  
        // Create frame  
        JFrame frame = new JFrame("JLabel Icon Demo");  
        frame.setSize(400, 300);  
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```

frame.setLayout(null);

// Load an icon (Make sure the image exists in your project folder)
Icon icon = new ImageIcon("img.jpg"); // Replace with your image file

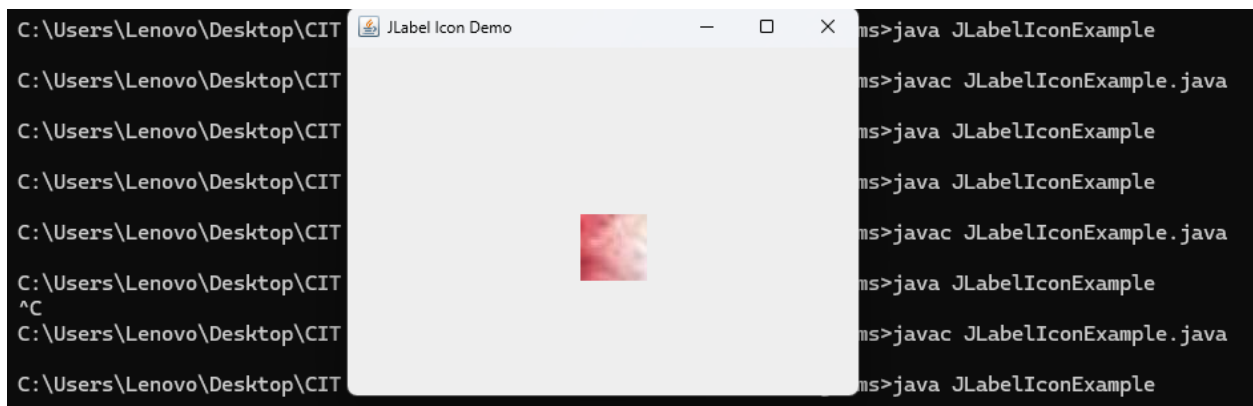
// Create JLabel using JLabel(Icon i)
JLabel label = new JLabel(icon);

// Set position and size
label.setBounds(100, 50, 200, 200);

// Add label to frame
frame.add(label);

// Make frame visible
frame.setVisible(true);
}
}

```



JButton

- A clickable **button** used to perform an action when pressed.
- Supports ActionListener for event handling.

Constructors:

JButton(Icon i)

JButton(String s)

JButton(String s, Icon i)

Here, s and i are the string and icon used for the button.

Example Program:

```
import javax.swing.*;

public class JButtonIconExample {

    public static void main(String[] args) {

        // Create a frame
        JFrame frame = new JFrame("JButton with Text and Icon");
        frame.setSize(400, 300);
        frame.setLayout(null);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Load an icon (make sure the file exists)
        Icon icon = new ImageIcon("img.jpg"); // Replace with your own image

        // Create JButton using JButton(String s, Icon i)
        JButton btn = new JButton("Click Me", icon);
```

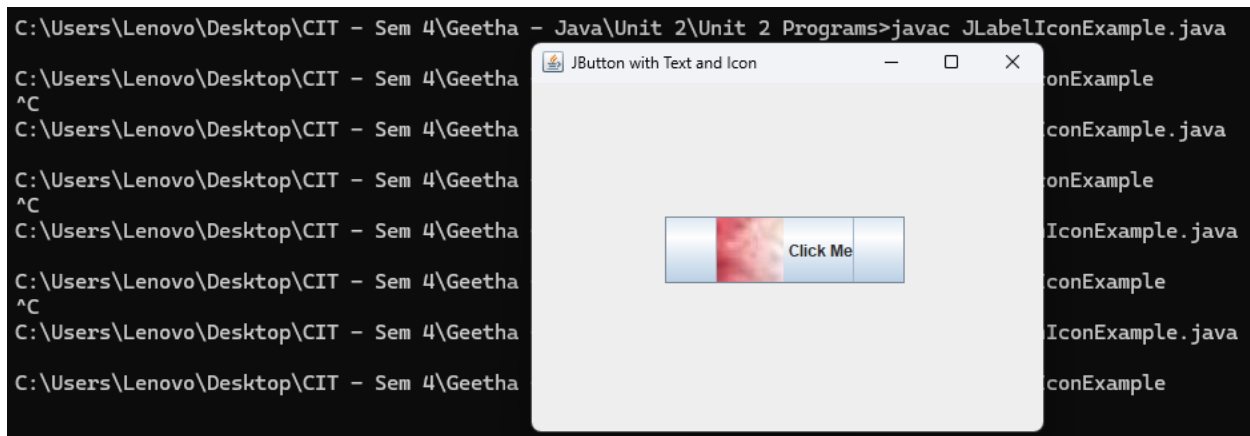
```

        // Set position and size
        btn.setBounds(100, 100, 180, 50);

        // Add button to frame
        frame.add(btn);

        // Make frame visible
        frame.setVisible(true);
    }
}

```



JTextField

- A **single-line** text input box.
- Used to get data like name, phone number, email, etc.

Constructors:

JTextField()

JTextField(int cols)

JTextField(String s, int cols)

`JTextField(String s)`

Here, s is the string to be presented, and cols is the number of columns in the text field.

Example Program:

```
import javax.swing.*;
```

```
import java.awt.event.*;
```

```
public class JTextFieldEventExample {  
    public static void main(String[] args) {
```

```
        // Create frame
```

```
        JFrame frame = new JFrame("JTextField with Event Handling");
```

```
        frame.setSize(400, 250);
```

```
        frame.setLayout(null);
```

```
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
        // Label
```

```
        JLabel lbl = new JLabel("Enter your name:");
```

```
        lbl.setBounds(50, 50, 150, 30);
```

```
        // TextField
```

```
        JTextField tf = new JTextField();
```

```
        tf.setBounds(180, 50, 150, 30);
```

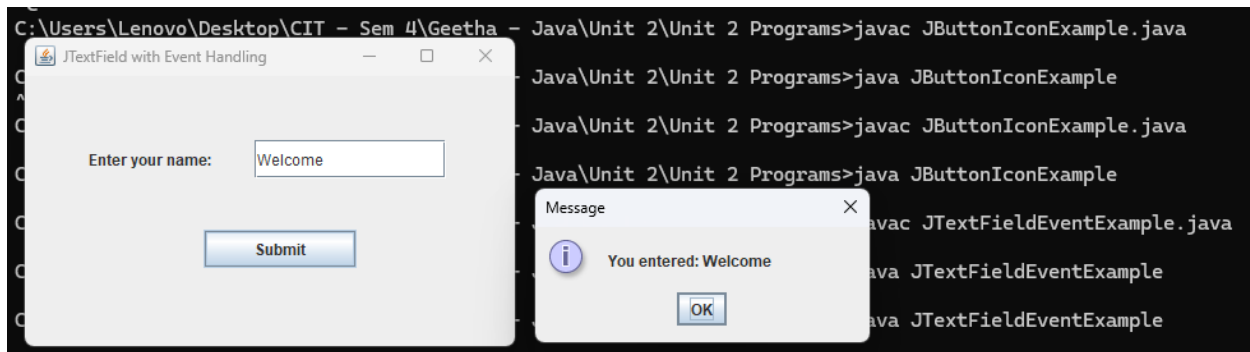
```
        // Button
```

```
JButton btn = new JButton("Submit");
btn.setBounds(140, 120, 120, 30);

// Event handling using ActionListener
btn.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String name = tf.getText();    // get text
        JOptionPane.showMessageDialog(frame, "You entered: " + name);
    }
});

// Add components
frame.add(lbl);
frame.add(tf);
frame.add(btn);

// Show frame
frame.setVisible(true);
}
}
```



JTextArea

- A multi-line text area used for long text input.
- Supports scrolling with JScrollPane.

Constructors:

JTextArea()

Creates an empty text area with default rows and columns.

JTextArea(String text)

Creates a text area with the specified initial text.

JTextArea(int rows, int columns)

Creates a text area with the given number of rows and columns.

JTextArea(String text, int rows, int columns)

Creates a text area with specified text, rows, and columns.

Example Program:

```
import javax.swing.*;
```

```
import java.awt.event.*;
```

```
public class JTextAreaEventExample {  
    public static void main(String[] args) {
```

```
// Frame
JFrame frame = new JFrame("JTextArea with Event Handling");
frame.setSize(500, 350);
frame.setLayout(null);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

// Label
JLabel lbl = new JLabel("Enter your message:");
lbl.setBounds(30, 20, 200, 30);

// TextArea
JTextArea ta = new JTextArea();
ta.setBounds(30, 60, 300, 150);

// Add scroll pane (optional but recommended)
JScrollPane sp = new JScrollPane(ta);
sp.setBounds(30, 60, 300, 150);

// Button
JButton btn = new JButton("Show Text");
btn.setBounds(150, 230, 120, 30);

// Event handling
btn.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
```

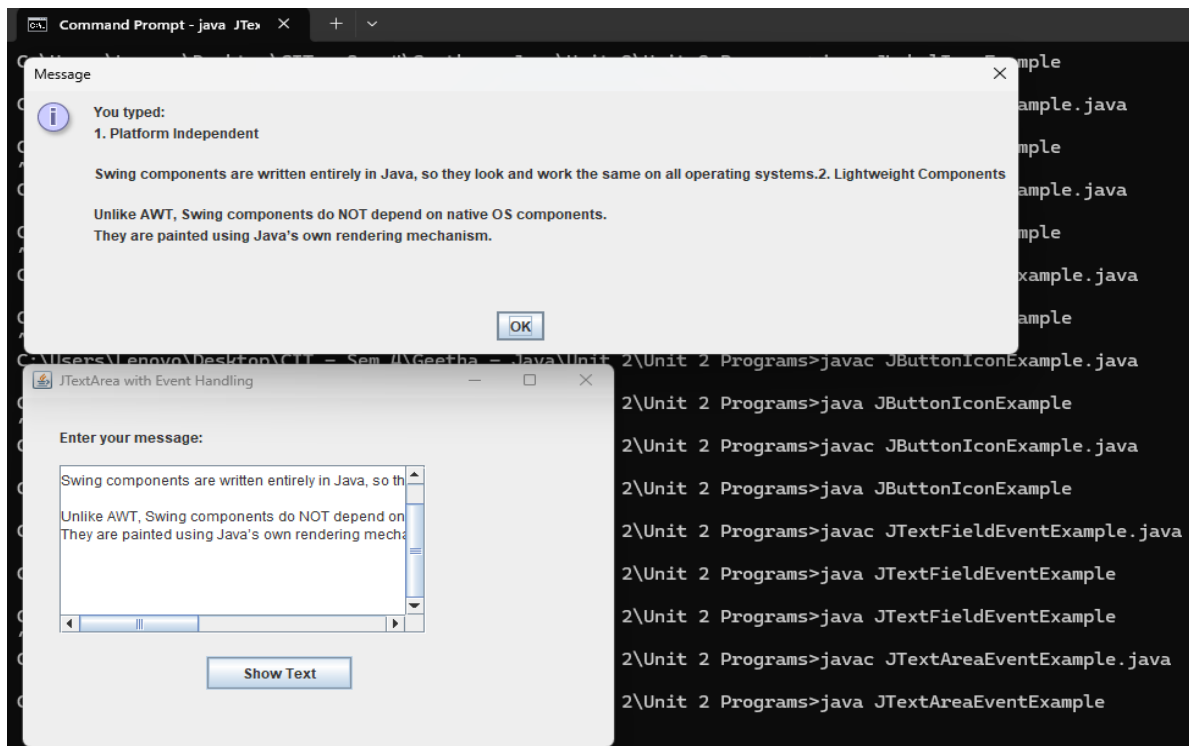
```

        String text = ta.getText(); // get text from JTextArea
        JOptionPane.showMessageDialog(frame, "You typed:\n" + text);
    }
});

// Add components
frame.add(lbl);
frame.add(sp);
frame.add(btn);

// Show frame
frame.setVisible(true);
}
}

```



JCheckBox

- Used for selecting **multiple** choices.
- Displays a small square box that can be checked/unchecked.

Constructors:

JCheckBox(Icon i)

JCheckBox(Icon i, boolean state)

JCheckBox(String s)

JCheckBox(String s, boolean state)

JCheckBox(String s, Icon i)

JCheckBox(String s, Icon i, boolean state)

Example Program:

```
import javax.swing.*;
```

```
import java.awt.event.*;
```

```
public class JCheckBoxIconExample {
```

```
    public static void main(String[] args) {
```

```
        // Frame
```

```
        JFrame frame = new JFrame("JCheckBox with Text, Icon, and State");
```

```
        frame.setSize(400, 300);
```

```
        frame.setLayout(null);
```

```
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
        // Load icon (make sure the image exists)
```

```
Icon icon = new ImageIcon("img.jpg"); // Replace with your image

// Create JCheckBox using (String s, Icon i, boolean state)
JCheckBox cb = new JCheckBox("Accept Terms", icon, true);
cb.setBounds(50, 50, 250, 40);

// Button
JButton btn = new JButton("Check Status");
btn.setBounds(120, 150, 140, 30);

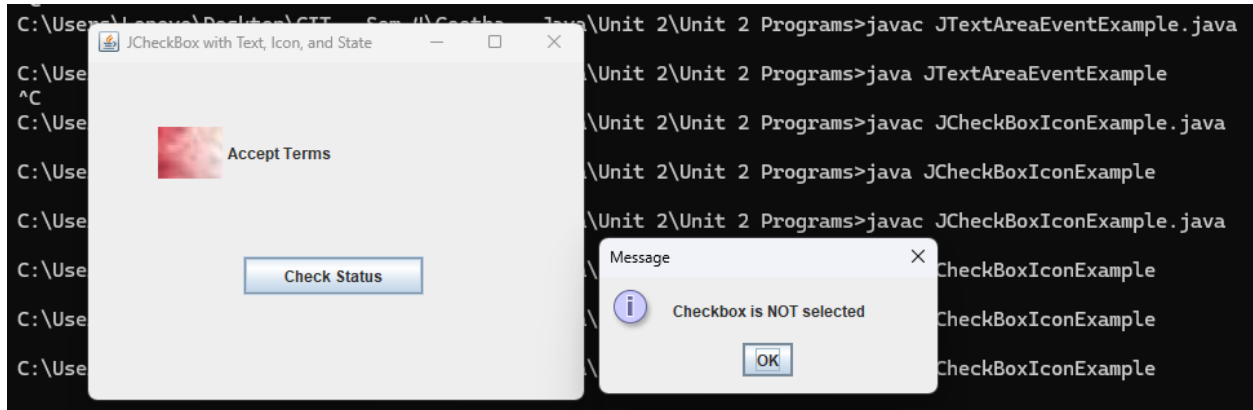
// Event Handling
btn.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if (cb.isSelected()) {
            JOptionPane.showMessageDialog(frame, "Checkbox is SELECTED");
        } else {
            JOptionPane.showMessageDialog(frame, "Checkbox is NOT
selected");
        }
    }
});

// Add components
frame.add(cb);
frame.add(btn);
```

```

        frame.setVisible(true);
    }
}

```



Radiobutton:

- Used when only **one option** must be selected from a group.

Constructors:

JRadioButton(Icon i)

JRadioButton(Icon i, boolean state)

JRadioButton(String s)

JRadioButton(String s, boolean state)

JRadioButton(String s, Icon i)

JRadioButton(String s, Icon i, boolean state)

Example Program:

```
import javax.swing.*;
import java.awt.event.*;

public class JRadioButtonExample {
    public static void main(String[] args) {

        // Frame
        JFrame frame = new JFrame("JRadioButton Example");
        frame.setSize(400, 300);
        frame.setLayout(null);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Load icon (replace with your own image)
        Icon icon = new ImageIcon("radio.png");

        // Create radio buttons using (String s, Icon i, boolean state)
        JRadioButton rb1 = new JRadioButton("Male", icon, true); // selected by
        default
        JRadioButton rb2 = new JRadioButton("Female", icon, false);

        // Set positions
        rb1.setBounds(50, 50, 200, 40);
        rb2.setBounds(50, 100, 200, 40);
```

```
// Grouping (only one option can be selected)
ButtonGroup group = new ButtonGroup();
group.add(rb1);
group.add(rb2);

// Button
JButton btn = new JButton("Submit");
btn.setBounds(120, 180, 120, 30);

// Event handling
btn.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String result = "";
        if (rb1.isSelected())
            result = "Selected: Male";
        else if (rb2.isSelected())
            result = "Selected: Female";

        JOptionPane.showMessageDialog(frame, result);
    }
});

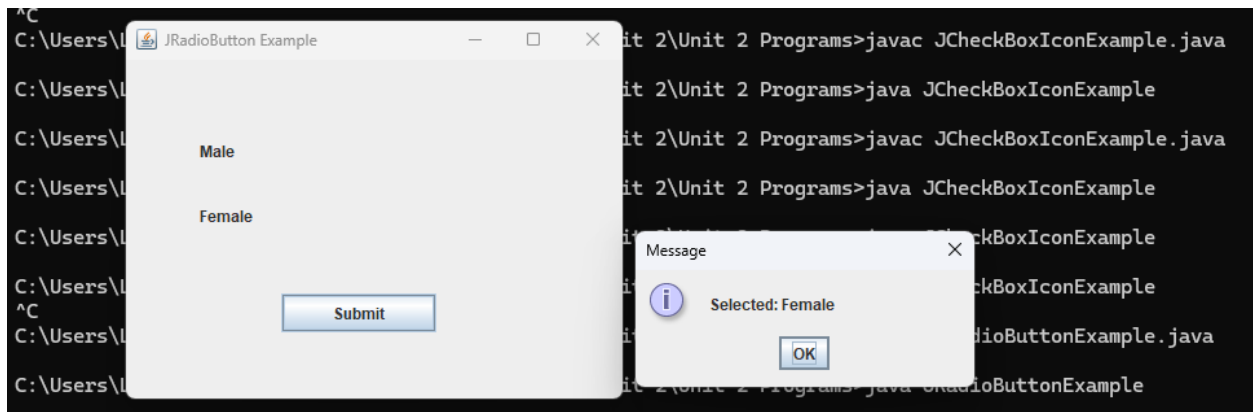
// Add components
frame.add(rb1);
frame.add(rb2);
```

```

        frame.add(btn);

        // Make visible
        frame.setVisible(true);
    }
}

```



JComboBox:

- A drop-down list that allows the user to select one item from several options.

Constructors:

```
JComboBox( )
```

Example Program:

```

import javax.swing.*.*;
import java.awt.event.*;

public class JComboBoxEventExample {
    public static void main(String[] args) {

```

```
// Frame
JFrame frame = new JFrame("JComboBox with Event Handling");
frame.setSize(400, 250);
frame.setLayout(null);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

// Label
JLabel lbl = new JLabel("Select a language:");
lbl.setBounds(50, 40, 150, 30);

// ComboBox with items
String languages[] = {"Java", "Python", "C++", "JavaScript"};
JComboBox<String> combo = new JComboBox<>(languages);
combo.setBounds(180, 40, 150, 30);

// Button
JButton btn = new JButton("Show Selected");
btn.setBounds(130, 120, 140, 30);

// Event handling (when button is clicked)
btn.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String selected = (String) combo.getSelectedItem();
        JOptionPane.showMessageDialog(frame, "You selected: " + selected);
    }
});
```

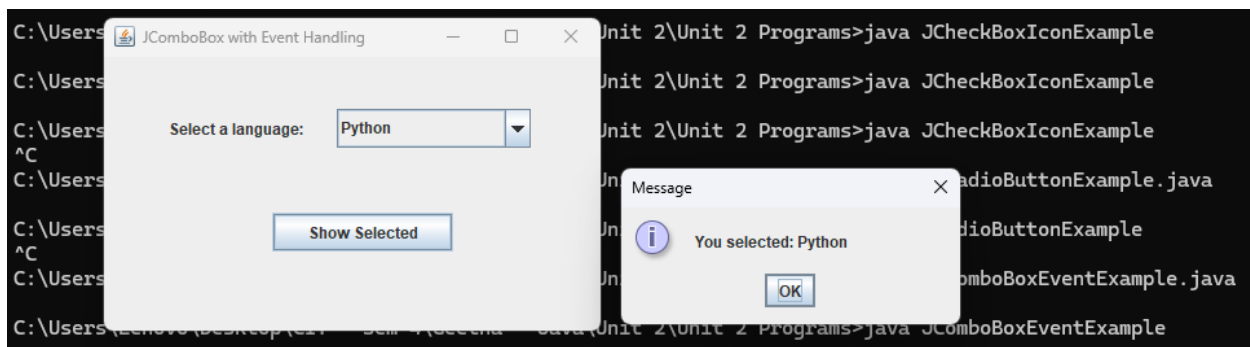
```

    }
});

// Add components
frame.add(lbl);
frame.add(combo);
frame.add(btn);

frame.setVisible(true);
}
}

```



JList:

JList class is useful to create a list which displays a list of items and allows the user to select one or more items.

Constructors:

- JList()
- JList(Object arr[])

- JList(Vector v)
 - ➔ Vector is a collection that can store multiple objects.
 - ➔ When the Vector is passed to JList, Swing automatically displays all its elements.
 - ➔ The list items are shown **vertically**, and the user can select one or more items.

Vector<String> v = new Vector<>(); //Vector – Data structures

v.add("Java");

v.add("Python");

v.add("C++");

JList<String> list = new JList<>(v);

Methods:

- getSelectedIndex() – returns selected item index
- getSelectedValue() – to know which item is selected in the list
- getSelectedIndices() – returns selected items into an array
- getSelectedValues() – returns selected items names into an array

Example Program:

```
import javax.swing.*;
```

```
import java.awt.event.*;
```

```
public class JListEventExample {
```

```
public static void main(String[] args) {

    // Frame
    JFrame frame = new JFrame("JList with Event Handling");
    frame.setSize(400, 300);
    frame.setLayout(null);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    // Label
    JLabel lbl = new JLabel("Select a Subject:");
    lbl.setBounds(20, 20, 150, 30);

    // JList items
    String subjects[] = {"Java", "Python", "C++", "HTML", "JavaScript"};
    JList<String> list = new JList<>(subjects);
    list.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

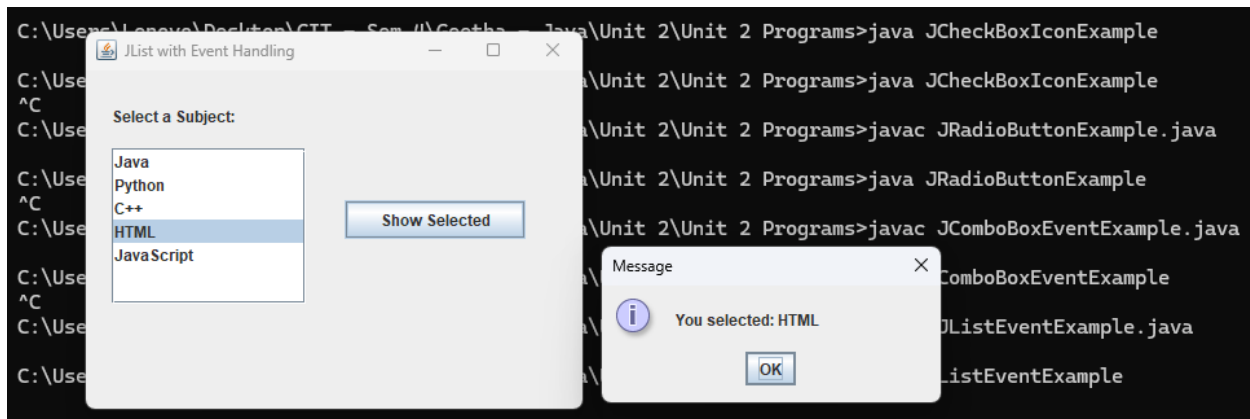
    // Scroll pane for list
    JScrollPane sp = new JScrollPane(list);
    sp.setBounds(20, 60, 150, 120);

    // Button
    JButton btn = new JButton("Show Selected");
    btn.setBounds(200, 100, 140, 30);
```

```
// Event Handling
btn.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String selected = list.getSelectedValue();
        if (selected != null) {
            JOptionPane.showMessageDialog(frame, "You selected: " +
selected);
        } else {
            JOptionPane.showMessageDialog(frame, "No item selected!");
        }
    }
});

// Add components
frame.add(lbl);
frame.add(sp);
frame.add(btn);

frame.setVisible(true);
}
}
```

JTable:

JTable is a Swing component used to display and edit data in a tabular form (rows and columns). It is similar to a spreadsheet where each cell can contain data.

JTable is widely used for:

- Displaying database records
- Showing tabular data
- Editing values in cells
- Creating forms, CRUD apps, admin dashboards

Example Program:

```
import javax.swing.*;
```

```
import javax.swing.event.*;
```

```
import java.awt.*;
```

```
public class JTableEventExample {
```

```
    public static void main(String[] args) {
```

```
        // Frame
```

```
        JFrame frame = new JFrame("JTable with Event Handling");
```

```
frame.setSize(500, 350);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setLayout(new BorderLayout());

// Table data
String[][] data = {
    {"101", "Alice", "Java"},
    {"102", "Bob", "Python"},
    {"103", "Charlie", "C++"},
    {"104", "David", "HTML"}
};

// Column names
String[] columns = {"ID", "Name", "Course"};

// Table
JTable table = new JTable(data, columns);

// Scroll pane for table
JScrollPane sp = new JScrollPane(table);

// Event Handling: Detect row selection
table.getSelectionModel().addListSelectionListener(new
ListSelectionListener() {
    public void valueChanged(ListSelectionEvent e) {
```

```

// Avoid firing twice
if (!e.getValueIsAdjusting()) {
    int row = table.getSelectedRow();
    if (row != -1) {
        String id = table.getValueAt(row, 0).toString();
        String name = table.getValueAt(row, 1).toString();
        String course = table.getValueAt(row, 2).toString();

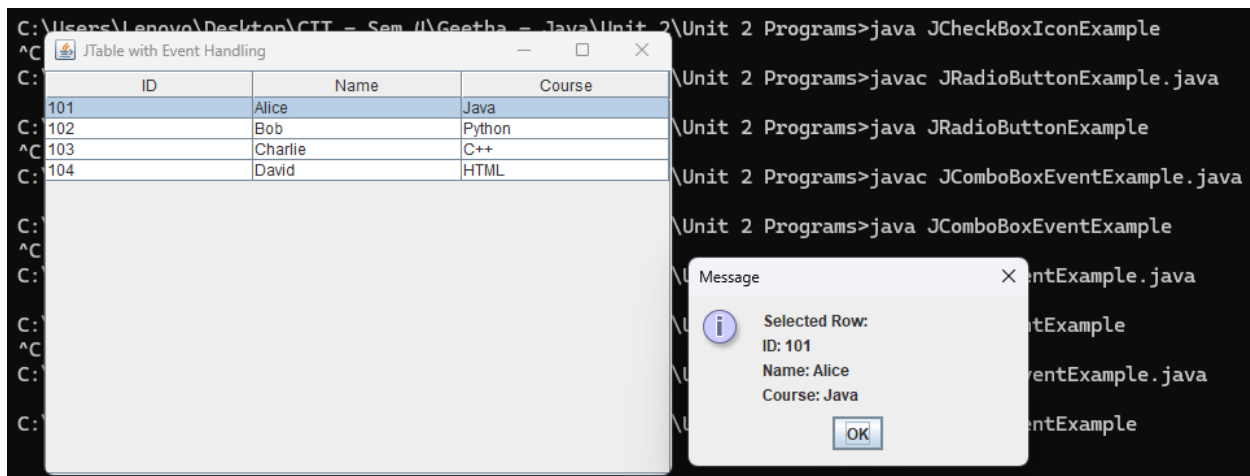
        JOptionPane.showMessageDialog(frame,
            "Selected Row:\nID: " + id +
            "\nName: " + name +
            "\nCourse: " + course);
    }
}

});

// Add components
frame.add(sp, BorderLayout.CENTER);

frame.setVisible(true);
}
}

```



JTree:

Tree is a graphical component that is used to represent hierarchical data in a structured and expandable format. The Swing component responsible for displaying hierarchical data is the JTree.

Java uses the DefaultMutableTreeNode class.

Example:

```

DefaultMutableTreeNode root = new DefaultMutableTreeNode("Root");
DefaultMutableTreeNode child1 = new DefaultMutableTreeNode("Child 1");
root.add(child1);
  
```

Example Program:

```

import javax.swing.*.*;
import javax.swing.event.*;
import javax.swing.tree.*;
import java.awt.*.*;

public class JTreeEventExample {
  
```

```
public static void main(String[] args) {

    // Frame
    JFrame frame = new JFrame("JTree with Event Handling");
    frame.setSize(400, 350);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setLayout(new BorderLayout());

    // Root node
    DefaultMutableTreeNode root = new
    DefaultMutableTreeNode("Courses");

    // Child nodes
    DefaultMutableTreeNode programming = new
    DefaultMutableTreeNode("Programming");

    DefaultMutableTreeNode web = new DefaultMutableTreeNode("Web
    Development");

    // Add leaf nodes
    programming.add(new DefaultMutableTreeNode("Java"));
    programming.add(new DefaultMutableTreeNode("Python"));
    programming.add(new DefaultMutableTreeNode("C++"));

    web.add(new DefaultMutableTreeNode("HTML"));
    web.add(new DefaultMutableTreeNode("CSS"));
    web.add(new DefaultMutableTreeNode("JavaScript"));
```

```

// Add child nodes to root
root.add(programming);
root.add(web);

// Create JTree
JTree tree = new JTree(root);

// Event Handling for node selection
tree.addTreeSelectionListener(new TreeSelectionListener() {
    public void valueChanged(TreeSelectionEvent e) {
        DefaultMutableTreeNode selectedNode =
            (DefaultMutableTreeNode)
tree.getLastSelectedPathComponent();

        if (selectedNode != null) {
            String node = selectedNode.toString();
            JOptionPane.showMessageDialog(frame,
                "You selected: " + node);
        }
    }
});

// Add tree inside scroll pane
JScrollPane sp = new JScrollPane(tree);

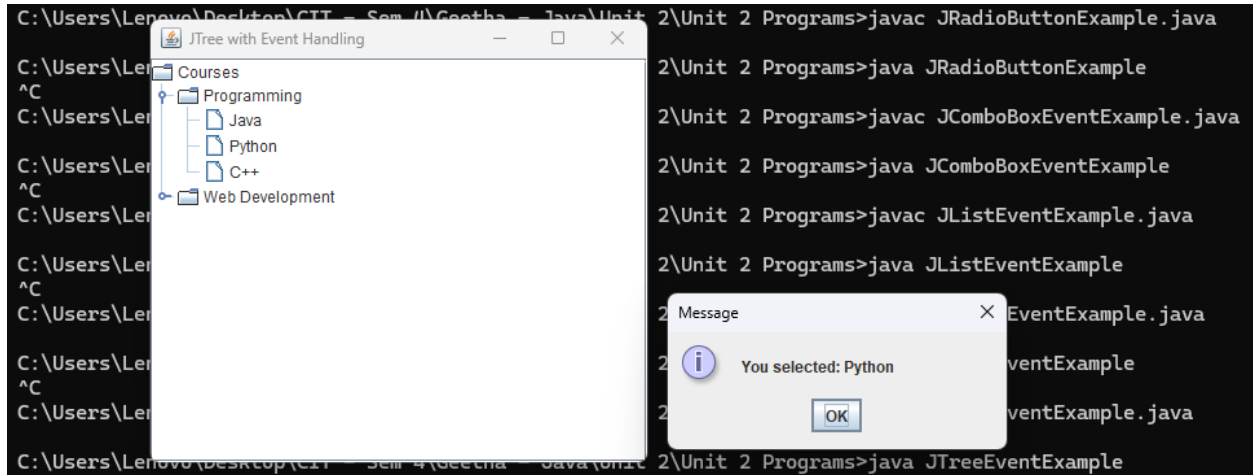
```

```

frame.add(sp, BorderLayout.CENTER);

frame.setVisible(true);
}
}

```



JSlider:

JSlider is a Swing component used to select a numeric value by sliding a knob along a track. It can be placed horizontally or vertically and can display tick marks and labels. It is commonly used for adjusting volume, brightness, size, or any numeric input.

Methods of JSlider:

Method	Description
getValue()	Returns the current slider value
setValue(int v)	Sets the slider to a specific value
setMinimum(int m)	Sets the minimum value
setMaximum(int m)	Sets the maximum value

Method	Description
setOrientation(int o)	Sets orientation: JSlider.HORIZONTAL or JSlider.VERTICAL
setMajorTickSpacing(int n)	Sets spacing for major tick marks
setMinorTickSpacing(int n)	Sets spacing for minor tick marks
setPaintTicks(boolean b)	Displays tick marks
setPaintLabels(boolean b)	Displays numbered labels
addChangeListener(ChangeListener l)	Handles change events
setEnabled(boolean b)	Enables/disables slider

Example Program:

```

import javax.swing.*;
import javax.swing.event.*;
import java.awt.*;

public class JSliderEventExample {
    public static void main(String[] args) {

        // Frame
        JFrame frame = new JFrame("JSlider with Event Handling");
        frame.setSize(400, 300);
        frame.setLayout(null);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

```



```
// Label for display
JLabel lbl = new JLabel("Value: 50");
lbl.setBounds(160, 30, 100, 30);

// Create JSlider (min=0, max=100, initial=50)
JSlider slider = new JSlider(0, 100, 50);
slider.setBounds(50, 100, 300, 50);

// Show ticks and labels (optional)
slider.setMajorTickSpacing(20);
slider.setMinorTickSpacing(5);
slider.setPaintTicks(true);
slider.setPaintLabels(true);

// Event Handling
slider.addChangeListener(new ChangeListener() {
    public void stateChanged(ChangeEvent e) {
        int value = slider.getValue();
        lbl.setText("Value: " + value);
    }
});

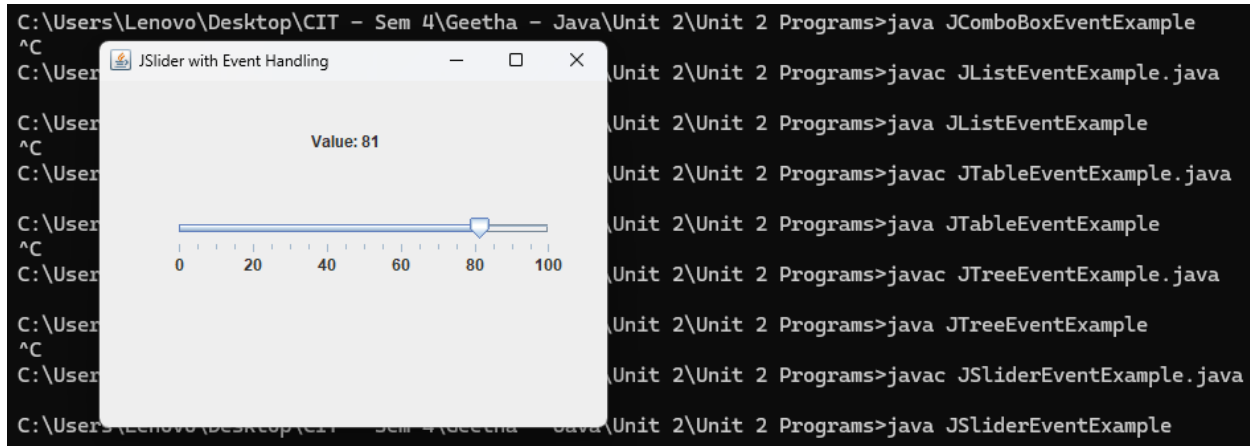
// Add components
frame.add(lbl);
```

```

frame.add(slider);

frame.setVisible(true);
}
}

```



JProgressBar:

JProgressBar is the main Swing component used to display progress.

Features:

- Can be horizontal or vertical
- Supports minimum and maximum values
- Can display percentage
- Supports indeterminate mode (unknown progress)
- Allows custom formatting

Constructors:

```
JProgressBar();
```

```
JProgressBar(int orientation);
```

```
JProgressBar(int min, int max);
```

```
JProgressBar(int orientation, int min, int max);
```

Orientation:

- JProgressBar.HORIZONTAL
- JProgressBar.VERTICAL

Important Methods:

Method	Description
setMinimum(int)	Sets starting value
setMaximum(int)	Sets ending value
setValue(int)	Updates progress
setStringPainted(true)	Shows percentage text
setIndeterminate(true)	Enables unknown progress animation
getValue()	Returns current value

Example Program:

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
public class ProgressBarDemo {
```

```
    public static void main(String[] args) {
```

```
        JFrame f = new JFrame("JProgressBar Demo");
```

```
        f.setSize(400, 200);
```

```
        JProgressBar pb = new JProgressBar(0, 100);
```

```
        pb.setValue(0);
```

```
pb.setStringPainted(true);
```

```
// Timer to increase progress
```

```
Timer t = new Timer(100, e -> {  
    int value = pb.getValue();  
    if (value < 100) {  
        pb.setValue(value + 1);  
    } else {  
        ((Timer)e.getSource()).stop();  
    }  
});
```

```
JButton startBtn = new JButton("Start");
```

```
startBtn.addActionListener(e -> t.start()); // event handling
```

```
f.setLayout(new FlowLayout());
```

```
f.add(pb);
```

```
f.add(startBtn);
```

```
f.setVisible(true);
```

```
}
```

```
}
```

```
C:\Users\Lenovo\Desktop\CTT - Sem 4\Geetha - Java\Unit 2\Unit 2 Programs>javac JTreeEventExample.java
C:\Users\Lenovo\Desktop\CTT - Sem 4\Geetha - Java\Unit 2\Unit 2 Programs>java JTreeEventExample
C:\Users\Lenovo\Desktop\CTT - Sem 4\Geetha - Java\Unit 2\Unit 2 Programs>javac JSliderEventExample.java
C:\Users\Lenovo\Desktop\CTT - Sem 4\Geetha - Java\Unit 2\Unit 2 Programs>java JSliderEventExample
C:\Users\Lenovo\Desktop\CTT - Sem 4\Geetha - Java\Unit 2\Unit 2 Programs>javac ProgressBarDemo.java
C:\Users\Lenovo\Desktop\CTT - Sem 4\Geetha - Java\Unit 2\Unit 2 Programs>java ProgressBarDemo
```

